二维多尺度问题正交分解方法及其程序实现

李东风

2030 年 6 月 5 日

目录

1 摘要 **2**

2 绪论 **3**

2.1 研究背景与意义 3

2.2 多尺度数值计算方法综述 3

2.3 局部正交分解（LOD）方法简介 4

2.4 本文主要研究内容与结构安排 4

3 椭圆型多尺度问题的数学描述 **4**

3.1 模型问题定义 4

3.2 标准有限元法的局限性 5

3.3 能量空间与有限元空间定义 6

4 二维 LOD 方法的理论框架 **6**

4.1 尺度的分解与拟插值算子 6

4.2 α-正交化与理想多尺度空间 7

4.3 理想 LOD 方法的误差分析 9

5 数值校正子的局部化技术 **9**

5.1 校正子的指数衰减特性 9

5.2 局部化校正算子与单元补片（Patches） 10

5.3 局部化正交分解方法（LOD）的离散格式 11

6 算法设计与 MATLAB 程序实现 **11**

6.1 算法总体流程与数据结构 11

6.2 核心模块实现细节 11

6.3 校正子问题的鞍点求解格式 13

6.4 粗网格刚度矩阵的组装与求解 14

7 数值算例与应用分析 **15**

7.1 非周期高反差系数问题算例 15

7.2 收敛性验证 15

7.3 结果可视化与解释 15

8 结论与展望 **18**

8.1 本文总结 18

8.2 未来研究方向 19

9 附录：主要源程序清单 **19**

1 摘要

本文系统研究了求解二维椭圆型多尺度扩散问题的局部正交分解（Localized Orthogonal Decomposition, LOD）方法及其 MATLAB 程序实现。针对传统有限元法在处理具有强非均匀、高震荡扩散系数问题时面临的”预渐近效应”困境，LOD 方法通过引入拟插值算子实现解空间的尺度分解，并利用 a-正交化构造具有微观信息编码能力的多尺度基函数。本文详细阐述了 LOD 方法的理论基础，包括尺度分解原理、校正算子的指数衰减特性及其局部化技术，并基于单元补片策略给出了实用的离散格式。在算法实现方面，本文基于红化加密网格，给出了拟插值算子矩阵表示、局部补片定位、鞍点格式校正子求解以及粗网格系统组装的全流程 MATLAB 程序。数值算例表明，LOD 方法对非周期高反差系数问题具有良好的适应性和最优收敛阶，其精度显著优于相同网格尺度下的标准有限元法。

关键词：多尺度问题；数值同质化；局部正交分解；拟插值算子；校正子；MATLAB 实现

2 绪论

2.1 研究背景与意义

现代工程与科学计算中广泛存在一类具有多尺度特征的物理问题。复合材料的热传导、多孔介质中的流体渗流、纤维网络的弹性分析等应用场景，其材料属性往往在微观尺度上呈现出剧烈的非均匀性和高震荡特征。以图 1.1 所示的纸张纤维网络模型为例，宏观力学性能的准确预测依赖于对微观纤维构型信息的有效提取，这种跨尺度的信息传递对数值模拟方法提出了严峻挑战。

从数学模型角度，这类问题通常归结为具有快速震荡系数的椭圆型偏微分方程：

$$-\operatorname{div}(A\nabla u) = f \quad \text{in } \Omega$$

其中扩散矩阵 A 在微观尺度上剧烈变化，且其变化频率与宏观计算域的特征尺度相比可能跨越数个数量级。传统的标准有限元法要求计算网格必须解析系数 A 的所有微观变化特征，这意味着需要采用与微观尺度相当的网格尺寸。当微观尺度远小于宏观计算域时，这将导致自由度数量呈指数级增长，超出当前计算能力。更为关键的是，即使在可计算范围内，当网格尺寸大于微观特征尺度时，有限元解将陷入所谓的”预渐近区间”（pre-asymptotic regime），其收敛行为完全失效，得到的宏观响应亦不可信。

上述困境催生了数值同质化（Numerical Homogenization）方法的发展。这类方法的核心思想在于：在不显式解析所有微观细节的前提下，构造能够”感知”并”编码”微观信息的粗尺度逼近空间，从而在可接受的计算代价下获得宏观尺度的精确预测。

2.2 多尺度数值计算方法综述

多尺度数值计算方法的发展可追溯至 20 世纪 60 年代开始的数学同质化理论。对于满足周期性假设和尺度分离条件的问题，同质化理论证明了当微观尺度参数 $\epsilon \to 0$ 时，原多尺度问题的解弱收敛于一个常系数”有效”方程的解。有效系数的构造依赖于在单胞上求解局部椭圆问题。这一理论框架直接催生了异质多尺度方法（Heterogeneous Multiscale Method, HMM）和多尺度有限元法（Multiscale Finite Element Method, MsFEM）等早期数值方法。

然而，周期性假设在真实物理问题中往往难以满足。许多先进材料的设计恰恰利用了非周期微结构带来的特殊宏观效应，如声子晶体中的负折射现象、无序介质中的 Anderson 局域化等。这些问题的共同特征是缺乏明确的尺度分离和周期性，使得经典同质化理论的直接推广面临根本性困难。

变分多尺度法（Variational Multiscale Method, VMS）提供了另一条理论路径。该方法将解空间分解为粗尺度有限元空间和一个由插值算子核函数定义的细尺度补空间，并通过在细尺度空间中求解局部问题来修正粗尺度基函数。早期 VMS 方法依赖于后验误差估计来指导自适应计算，缺乏先验的理论收敛性保证。

局部正交分解（Localized Orthogonal Decomposition, LOD）方法的提出标志着这一领域的重大突破。该方法在保留 VMS 基本框架的基础上，通过引入与椭圆算子相关联的能量内积实现尺度的 a-正交化分解，并严格证明了

所得校正子的指数衰减特性。这一定性结果为局部化近似提供了坚实的理论支撑，使得 LOD 方法能够在不依赖周期性假设的前提下获得最优收敛阶。自此，以 LOD 方法为代表的新一代数值同质化方法成功地摆脱了经典同质化理论的结构性约束。

2.3 局部正交分解（LOD）方法简介

LOD 方法的核心思想可概括为三个层次。首先，利用拟插值算子 $\mathcal{J}_H$ 将解空间 V 分解为粗尺度有限元空间 V_H 和细尺度空间 $W = \ker(\mathcal{J}_H)$。这一分解在标准 H^1 内积下是稳定的，但在问题相关的能量内积 $a(\cdot,\cdot)$ 下并无正交性。其次，保持细尺度空间 W 不变，构造其在 a-内积下的正交补空间 V_H^{ms}。这一正交化过程等价于对每个粗尺度基函数 Λ_z 施加一个细尺度校正 $\phi_z \in W$，使得校正后的基函数 $\Lambda_z - \phi_z$ 能够自动" 感知" 扩散系数的微观变化。第三，利用校正子的指数衰减特性，将全局校正问题截断为在局部单元补片 $\mathsf{N}^\ell(T)$ 上独立求解的子问题，从而实现计算复杂度的线性缩放。

LOD 方法的突出优势体现在三个方面：其一，适用范围广，不依赖周期性、尺度分离或系数的光滑性假设；其二，具有完善的理论保证，能量模和 L^2 模下的误差估计与标准有限元法在光滑问题上的收敛阶相当；其三，算法高度可并行化，各局部校正子问题完全独立求解，天然适合大规模并行计算环境。

2.4 本文主要研究内容与结构安排

本文围绕二维椭圆型多尺度问题的 LOD 方法及其 MATLAB 程序实现展开系统研究，共分为七章。

第一章为绪论，阐述研究背景与意义，综述多尺度数值计算方法的发展脉络，并简介 LOD 方法的基本思想。第二章给出椭圆型多尺度问题的数学描述，并通过数值实验揭示标准有限元法的局限性。第三章系统构建 LOD 方法的理论框架，包括拟插值算子的构造、尺度的 a-正交化分解以及理想方法的误差分析。第四章研究数值校正子的局部化技术，证明校正子的指数衰减特性，并给出基于单元补片的局部化离散格式。第五章聚焦算法设计与 MATLAB 程序实现，详细阐述网格生成、拟插值算子矩阵表示、局部补片定位、鞍点格式求解以及粗网格系统组装等核心模块。第六章通过非周期高反差系数算例验证方法的收敛性和有效性。第七章总结全文并展望未来研究方向。

---

3 椭圆型多尺度问题的数学描述

3.1 模型问题定义

设 $\Omega \subset \mathbb{R}^2$ 为具有 Lipschitz 边界 $\partial\Omega$ 的有界多边形区域。考虑如下具有强非均匀扩散系数的椭圆型边值问题：

$$\begin{cases} -\operatorname{div}(A(x)\nabla u(x)) = f(x) & \text{in } \Omega, \\ u(x) = 0 & \text{on } \partial\Omega, \end{cases}$$

其中 $f \in L^2(\Omega)$ 为给定的源项，扩散矩阵 $A \in L^\infty(\Omega; \mathbb{R}^{2\times 2}_{\text{sym}})$ 满足一致椭圆性条件：存在正常数 α, β 使得

$$0 < \alpha \le \operatorname{ess\,inf}_{x\in\Omega} \inf_{v\in\mathbb{R}^2\setminus\{0\}} \frac{(A(x)v)\cdot v}{v\cdot v} \le \operatorname{ess\,sup}_{x\in\Omega} \sup_{v\in\mathbb{R}^2\setminus\{0\}} \frac{(A(x)v)\cdot v}{v\cdot v} \le \beta < \infty.$$

该问题的弱形式为：求 $u \in V := H_0^1(\Omega)$，使得

$$a(u,v) = \int_\Omega f v \,\mathrm{d}x, \quad \forall v \in V,$$

其中双线性型 $a(\cdot,\cdot)$ 定义为

$$a(u,v) := \int_\Omega (A\nabla u)\cdot \nabla v \,\mathrm{d}x.$$

由 Lax-Milgram 引理，该问题存在唯一解，且满足先验估计

$$\|A^{1/2}\nabla u\|_{L^2(\Omega)} \le \frac{\operatorname{width}(\Omega)}{\pi\sqrt{\alpha}} \|f\|_{L^2(\Omega)}.$$

3.2 标准有限元法的局限性

为阐释标准有限元法处理多尺度问题时的根本性困难，考虑一个具有代表性的一维算例。取 $\Omega = (0,1)$，源项 $f \equiv 1$，扩散系数为

$$A_\epsilon(x) = \left(2 + \cos\left(2\pi\frac{x}{\epsilon}\right)\right)^{-1}, \quad \epsilon = 2^{-5}.$$

采用 P_1 有限元在均匀网格上求解该问题，网格尺寸 h 从 2^{-1} 逐步加密至 2^{-7}。数值结果揭示了两个截然不同的收敛阶段。

当 $h > \epsilon$ 时，网格无法分辨系数的微观震荡。有限元解的局部平均效应使得扩散系数被隐式地替换为局部算术平均，而非正确的调和平均。如图 2.1 所示，这一阶段的有限元解不仅无法捕捉解的微观震荡特征，其宏观幅值也严重偏离真实解。有限元法陷入了”预渐近区间”，收敛性完全失效。

只有当 $h \ll \epsilon$ 时，网格能够逐点解析系数变化，有限元解才开始呈现出标准的二次收敛行为。从预渐近区间到渐近区间的转变以 $h \approx \epsilon$ 为特征尺度。在许多实际多尺度问题中，ϵ 通常极其微小，使得渐近区间的到达需要天文数字的计算自由度，这在工程实践中往往无法实现。

上述分析表明，标准有限元法在多尺度问题上的根本困境不在于逼近能力的理论缺陷，而在于其逼近空间与问题物理本质的不匹配。标准有限元基函数是与问题无关的通用多项式，无法感知系数的微观变化。解决这一困境的关键在于构造一种”问题适配”的离散空间，其基函数能够自动编码扩散系数的微观信息。

3.3 能量空间与有限元空间定义

设 $\mathcal{T}_H$ 为区域 Ω 的一个正则单纯形剖分（二维情形为三角形网格），网格尺寸参数 H 表示最大单元直径。网格满足形状正则性条件，即存在常数 $\gamma > 0$ 使得

$$\max_{T\in\mathcal{T}_H} \frac{H_T}{\operatorname{diam}(B_T)} \le \gamma,$$

其中 B_T 为单元 T 内的最大内切球。定义连续的分片线性有限元空间

$$V_H := \{v \in H_0^1(\Omega) : v|_T \in P_1(T),\ \forall T \in \mathcal{T}_H\}.$$

记 $\mathcal{N}_H$ 为 $\mathcal{T}_H$ 的顶点集合，$\mathcal{N}_H^{\text{int}}$ 为内部顶点子集。对于每一内部顶点 $z \in \mathcal{N}_H^{\text{int}}$，记 $\Lambda_z \in V_H$ 为相应的节点基函数（“帽函数”），满足 $\Lambda_z(z) = 1$ 且 $\Lambda_z(y) = 0$ 对所有 $y \neq z$ 成立。

在理论分析中，我们还需引入一个足够精细的参考网格 $\mathcal{T}_h$，其网格尺寸 $h \ll H$ 能够充分解析扩散系数 A 的所有微观变化。相应的有限元空间记为 V_h。在实际计算中，$\mathcal{T}_h$ 仅在局部补片上使用，而非全局构造。

---

4 二维 LOD 方法的理论框架

4.1 尺度的分解与拟插值算子

LOD 方法的理论出发点是解空间 V 的尺度分解。与一维情形不同，二维空间中节点插值算子对 H^1 函数无定义，因此需引入拟插值算子作为替代。本文采用如下投影型拟插值算子：

$$\mathcal{J}_H := E_H \circ \Pi_H^{\text{dg}},$$

其中 $\Pi_H^{\text{dg}} : V \to P_1(\mathcal{T}_H)$ 为 L^2 正交投影到间断分片线性函数空间，E_H 为取相邻单元函数值的加权平均算子。具体地，对于 $v \in P_1(\mathcal{T}_H)$ 和内部顶点 z，

$$(E_H(v))(z) = \sum_{T\in\mathcal{T}_H : z\in T} \frac{|T|}{|\mathsf{N}(z)|} v|_T(z),$$

边界顶点处显式置零以满足 Dirichlet 边界条件。该算子具有以下基本性质：

1. **投影性**：$\mathcal{J}_H \circ \mathcal{J}_H = \mathcal{J}_H$。
2. **稳定性与逼近性**：存在常数 C_I 仅依赖于网格正则性参数 γ，使得对任意 $v \in V$ 和 $T \in \mathcal{T}_H$，

$$H_T^{-1}\|v-\mathcal{I}_H v\|_{L^2(T)}+\|\nabla\mathcal{I}_H v\|_{L^2(T)}\le C_I\|\nabla v\|_{L^2(\mathsf{N}(T))},$$

其中 $\mathsf{N}(T)$ 为与 T 相邻的单元组成的补片。

定义细尺度空间 W 为拟插值算子 $\mathcal{I}_H$ 的核：

$$W:=\{w\in V:\mathcal{I}_H w=0\}=\ker(\mathcal{I}_H).$$

任意 $v\in V$ 可唯一分解为

$$v=\mathcal{I}_H v+(1-\mathcal{I}_H)v,\quad \mathcal{I}_H v\in V_H,\ (1-\mathcal{I}_H)v\in W.$$

这一分解在 H^1 范数下是稳定的。图1展示了典型多尺度解的尺度分解效果：粗尺度部分 $\mathcal{I}_H u$ 捕捉了解的宏观趋势，而细尺度部分 $(1-\mathcal{I}_H)u$ 包含了所有微观震荡信息。

```
knitr::include_graphics("figures/fig3-1-diffusion-coefficient.png")
```

4.2 a-正交化与理想多尺度空间

上述分解在标准 H^1 内积下虽具有稳定性，但与问题相关的能量内积 $a(\cdot,\cdot)$ 并无正交性。为获得对多尺度问题的最优逼近性质，需对分解进行 a-正交化处理。

保持细尺度空间 W 不变，定义其在 a-内积下的正交补空间：

$$V_H^{\mathrm{ms}}:=\{v\in V:a(v,w)=0,\ \forall w\in W\}.$$

由此得到解空间 V 的 a-正交分解：

$$V=V_H^{\mathrm{ms}}\oplus W,\quad a(V_H^{\mathrm{ms}},W)=0.$$

定义校正算子 $\mathcal{C}:V\to W$ 为到 W 的 a-正交投影，即对任意 $v\in V$，$\mathcal{C}v\in W$ 满足

$$a(\mathcal{C}v,w)=a(v,w),\quad \forall w\in W.$$

容易验证，$1-\mathcal{C}$ 是到 V_H^{ms} 的 a-正交投影，且

$$V_H^{\mathrm{ms}}=(1-\mathcal{C})V_H.$$

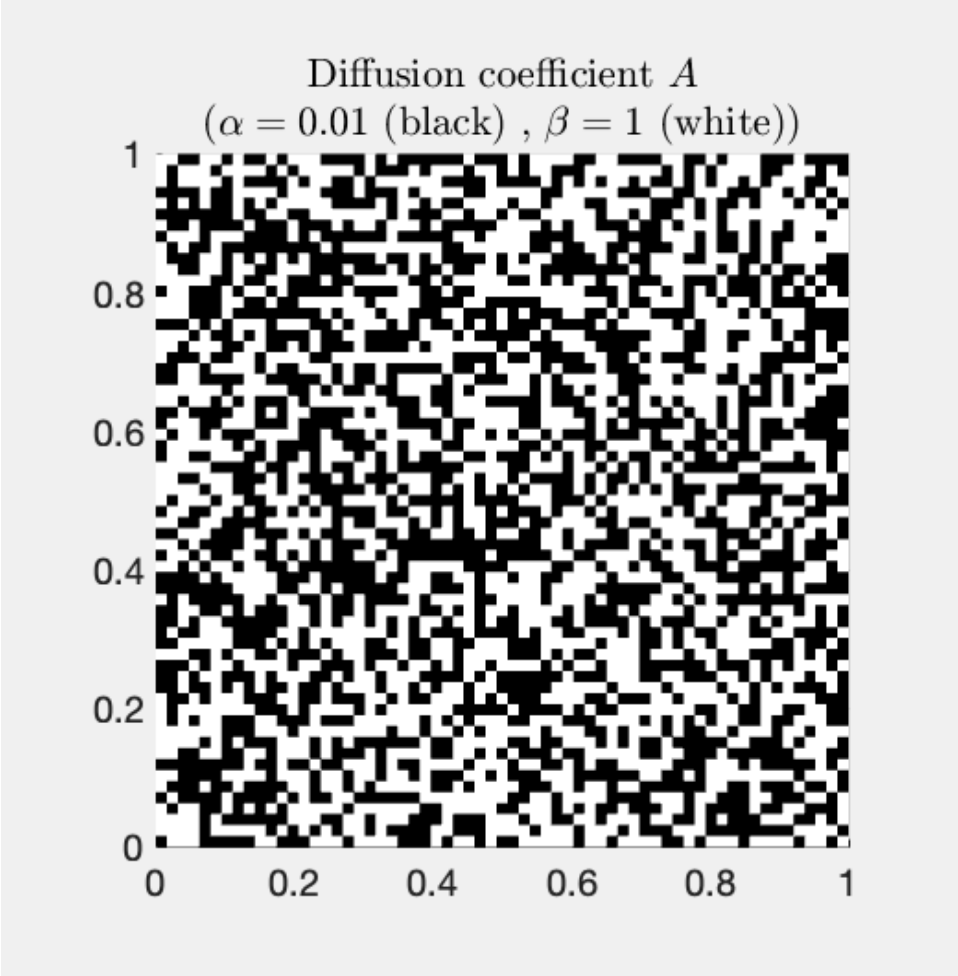

图 1: 扩散系数随机分布示意（图内标题为英文）

V_H^{ms} 的一组基可通过将校正算子作用于标准有限元节点基函数得到：

$$\Lambda_z^{\mathrm{ms}} := (1-\mathcal{C})\Lambda_z, \quad z \in \mathcal{N}_H^{\mathrm{int}}.$$

这些多尺度基函数保留了标准基函数的 Lagrange 性质，即 $\Lambda_z^{\mathrm{ms}}(y) = \delta_{zy}$，但其形状能够自动适应扩散系数 A 的局部变化。图 2.12 展示了校正前后基函数的显著差异：标准基函数是光滑的帽函数，而多尺度基函数在微观尺度上呈现出与 A 相同的震荡模式。

4.3 理想 LOD 方法的误差分析

基于多尺度空间 V_H^{ms} 的 Galerkin 投影定义了理想 LOD 方法：求 $u_H^{\mathrm{ms}} \in V_H^{\mathrm{ms}}$，使得

$$a(u_H^{\mathrm{ms}}, v) = \int_\Omega f v \,\mathrm{d}x, \quad \forall v \in V_H^{\mathrm{ms}}.$$

由 Galerkin 正交性，误差 $e := u - u_H^{\mathrm{ms}}$ 满足 $a(e,v) = 0$ 对所有 $v \in V_H^{\mathrm{ms}}$ 成立。由于 V_H^{ms} 与 W 的 a-正交性，误差必然属于细尺度空间 W，从而

$$\mathcal{I}_H u = \mathcal{I}_H u_H^{\mathrm{ms}}.$$

结合拟插值算子的逼近性质与对偶论证，可得如下误差估计：

$$\|u - u_H^{\mathrm{ms}}\|_{L^2(\Omega)} \le \frac{C_I^2}{\alpha} H^2 \|f\|_{L^2(\Omega)},$$
$$\|A^{1/2}\nabla(u - u_H^{\mathrm{ms}})\|_{L^2(\Omega)} \le \frac{C_I}{\sqrt{\alpha}} H \|f\|_{L^2(\Omega)}.$$

值得强调的是，上述估计的成立不依赖于解 u 的高阶正则性或扩散系数 A 的光滑性。常数仅取决于下界 α 和网格正则性参数，与系数的震荡频率和幅度（上限 β 除外）无关。这一”鲁棒收敛”特性是 LOD 方法区别于传统有限元法的核心优势。

---

5 数值校正子的局部化技术

5.1 校正子的指数衰减特性

理想 LOD 方法要求在每个粗网格节点处求解一个全局的校正子问题，其计算代价与细网格自由度相当，在实际中无法接受。然而，校正算子 $\mathcal{C}$ 具有一个至关重要的数学性质——指数衰减。

定理 4.1（校正子的指数衰减）设 $\phi \in W$ 是如下变分问题的解：

$$a(\phi, w) = \int_T (A\nabla v) \cdot \nabla w \, \mathrm{d}x, \quad \forall w \in W,$$

其中 $T \in \mathcal{T}_H$ 为某一单元，$v \in V$。则存在常数 $c > 0$ 仅依赖于网格正则性，使得对任意 $\ell \in \mathbb{N}_0$，

$$\|A^{1/2}\nabla\phi\|_{L^2(\Omega\setminus\mathsf{N}^\ell(T))} \leq \sqrt{2}\exp\left(-c\frac{\alpha}{\beta}\ell\right)\|A^{1/2}\nabla v\|_{L^2(T)}.$$

定理的证明基于能量方法和适当的截断函数技巧。其核心物理直观是：椭圆算子的 Green 函数在异质介质中虽然呈现复杂的局部震荡，但其”宏观包络”仍以指数速率衰减。图 4.1 以对数色标展示了校正函数的快速空间衰减行为，与理论预测高度吻合。

5.2 局部化校正算子与单元补片（Patches）

基于指数衰减性质，可将全局校正问题截断到局部补片上求解。对每一粗网格单元 $T \in \mathcal{T}_H$ 和局部化参数 $\ell \in \mathbb{N}_0$，定义局部细尺度空间

$$W^\ell(T) := \{w \in W : \mathrm{supp}(w) \subseteq \mathsf{N}^\ell(T)\}.$$

局部校正算子 $\mathcal{C}_T^\ell : V \to W^\ell(T)$ 定义为：$\mathcal{C}_T^\ell v \in W^\ell(T)$ 满足

$$a(\mathcal{C}_T^\ell v, w) = \int_T (A\nabla v) \cdot \nabla w \, \mathrm{d}x, \quad \forall w \in W^\ell(T).$$

将所有单元的局部校正叠加，得到全局局部化校正算子

$$\mathcal{C}^\ell := \sum_{T\in\mathcal{T}_H} \mathcal{C}_T^\ell.$$

定理 4.2（局部化误差估计）对任意 $v \in V$ 和 $\ell \in \mathbb{N}_0$，

$$\|A^{1/2}\nabla(\mathcal{C} - \mathcal{C}^\ell)v\|_{L^2(\Omega)} \leq C(\ell^{d/2} + 1)\frac{\beta}{\alpha}\exp\left(-c\frac{\alpha}{\beta}\ell\right)\|A^{1/2}\nabla v\|_{L^2(\Omega)}.$$

该估计表明，局部化误差以指数速率随补片尺寸 ℓ 增大而衰减。当取 $\ell \approx |\log H|$ 时，局部化误差与粗网格离散误差同阶，从而在保持最优收敛率的前提下将计算复杂度降至 $\mathcal{O}(N_H(\log N_H)^d)$。

5.3 局部化正交分解方法（LOD）的离散格式

用 $\mathcal{C}^\ell$ 替换理想校正算子 $\mathcal{C}$，得到实用的 LOD 方法。定义局部化多尺度空间

$$V_{H,\ell}^{\mathrm{ms}} := (1-\mathcal{C}^\ell)V_H.$$

定义 4.1（LOD 方法） 求 $u_{H,\ell}^{\mathrm{ms}} \in V_{H,\ell}^{\mathrm{ms}}$，使得

$$a(u_{H,\ell}^{\mathrm{ms}}, v) = \int_\Omega fv\,\mathrm{d}x, \quad \forall v \in V_{H,\ell}^{\mathrm{ms}}.$$

在细尺度 h 上进一步离散局部校正子问题，得到完全离散的 LOD 方法。其误差估计为：

$$\|u_h - u_{H,\ell,h}^{\mathrm{ms}}\|_{L^2(\Omega)} \lesssim (H+\exp(-c\ell))^2, \quad \|u_h - u_{H,\ell,h}^{\mathrm{ms}}\|_{H^1(\Omega)} \lesssim H+\exp(-c\ell).$$

当 $\ell \approx |\log H|$ 时，收敛阶恢复为 H^2（L^2 模）和 H（H^1 模）。

---

6 算法设计与 MATLAB 程序实现

6.1 算法总体流程与数据结构

LOD 方法的完整计算流程可分为离线阶段和在线阶段。离线阶段负责构造多尺度基函数并组装粗网格刚度矩阵，其计算量占主导但高度可并行。在线阶段仅需在粗网格上求解约 N_H 个自由度的线性系统。

程序采用 MATLAB 实现，核心数据结构如下：

- **网格结构**：`Th.p`（节点坐标，$N_h \times 2$ 矩阵）、`Th.t`（单元-节点关联，$N_{Th} \times 3$ 矩阵）、`Th.Dnodes`（Dirichlet 边界节点索引）。
- **邻接关系**：`Ivt`（节点-单元关联矩阵）、`Itt`（单元-单元邻接矩阵，通过 `Ivt'*Ivt` 计算）。
- **投影算子**：`P1`（连续 P_1 延拓）、`P1dg`（间断 P_1 延拓）、`P0`（分片常数延拓）。

6.2 核心模块实现细节

6.2.1 网格生成与加密

初始网格为 $(0,1)^2$ 上由两个直角三角形组成的标准剖分。通过 `refineMesh` 函数执行红化加密：将每个三角形单元的中点两两相连，分割为四个全等子三角形。加密过程中同时构造粗-细网格间的延拓与限制算子。

对应主程序中的网格初始化与双层加密实现如下：

```
% initial triangulation
p = [0 0; 1 0; 1 1; 0 1];
t = [1 2 4; 2 3 4];
T0 = struct('t',t,'p',p,'Dnodes',[1;2;3;4]);

% LOD parameters
d = 2;
Hlevel = 2;
hlevel = 8;
ell = 1;

% coarse and fine mesh
[TH] = refineMesh(T0,Hlevel);
[Th,P1,R1,P0,P1dg] = refineMesh(TH,hlevel-Hlevel);
NH = size(TH.p,1);   NTH = size(TH.t,1);
Nh = size(Th.p,1);   NTh = size(Th.t,1);
```

6.2.2 拟插值算子的矩阵表示

拟插值算子 $\mathcal{J}_H = E_H \circ \Pi_H^{\mathrm{dg}}$ 的矩阵表示 IH 通过以下步骤构造:

1. 计算间断分片线性函数的 L^2 质量矩阵 Mhdg 及其粗网格版本。
2. 组装 Π_H^{dg} 矩阵: PiHdg = B * (P1dg' * Mhdg) * cg2dgh, 其中 B 为粗间断质量矩阵的块对角逆。
3. 组装 E_H 矩阵: EH = spdiags(1./sum(cg2dgH,1)', 0, NH, NH) * cg2dgH'。
4. 合成: IH = EH * PiHdg。

对应代码片段如下:

```
Nhdg = numel(Th.t);
NHdg = numel(TH.t);
dgHidx = reshape(1:NHdg,d+1,[]).';
dghidx = reshape((1:Nhdg),d+1,[])';
nngH = accumarray(TH.t(:),1,[NH,1]); nngH(TH.Dnodes) = 0;
nngh = accumarray(Th.t(:),1,[Nh,1]); nngh(Th.Dnodes) = 0;
cg2dgh = sparse(1:Nhdg,reshape(Th.t.',1,[]),1,Nhdg,Nh);
Mhdg = kron(spdiags(computeArea(Th),0,NTh,NTh),[2 1 1;1 2 1;1 1 2]./12);
B = kron(spdiags(1./computeArea(TH),0,NTH,NTH),[9 -3 -3;-3 9 -3;-3 -3 9]);
PiHdg = B*((P1dg.'*Mhdg)*cg2dgh);
```

```
cg2dgH = sparse(1:NHdg,reshape(TH.t.',1,[]),1,NHdg,NH);
EH = spdiags(1./sum(cg2dgH,1).',0,NH,NH)*cg2dgH.';
IH = EH*PiHdg;
```

6.2.3 局部补片定位算法

补片矩阵 patch 通过邻接矩阵的幂次迭代计算:

```
Ivt = sparse(TH.t,repmat((1:NTH).',1,d+1),1,NH,NTH);
Itt = spones(Ivt.'*Ivt);
patch = speye(NTH);
for k = 1:ell
    patch = Itt * patch;
end
patch = spones(patch) + speye(NTH);
```

patch(:,k) 中的非零项标识了第 k 单元 ℓ-补片包含的所有粗单元。

6.3 校正子问题的鞍点求解格式

校正子 $\phi_i \in W_h^\ell(T)$ 满足约束条件 $\mathcal{I}_H\phi_i = 0$。直接构造 $W_h^\ell(T)$ 的基函数较为复杂，程序改用鞍点格式求解约束最小化问题:

$$\begin{bmatrix} S_{\mathrm{ph}} & \mathrm{IHp}^T \\ \mathrm{IHp} & 0 \end{bmatrix} \begin{bmatrix} \phi_i \\ \mu \end{bmatrix} = \begin{bmatrix} \mathrm{rhs}_i \\ 0 \end{bmatrix}.$$

程序通过 Schur 补消去 ϕ_i:

```
CT = cell(1,NTH);
for k = 1:NTH
    tpH = logical(patch(:,k));
    dofpH = find(accumarray(reshape(TH.t(tpH,:),[],1),1,[NH 1]));
    dofpH = dofpH(nngH(dofpH)>0);

    tph = logical(P0*patch(:,k));
    nngph = accumarray(reshape(Th.t(tph,:),[],1),1,[Nh 1]);
    dofph = find(nngph);
```

```
    dofph = dofph(nngph(dofph)==nngh(dofph));

    dofphdg = reshape(dghidx(tph,:).',[],1);
    cg2dghk = cg2dgh(dofphdg,dofph);
    Sph = (cg2dghk.'*Shdg(dofphdg,dofphdg))*cg2dghk;
    tTh = logical(P0*patch(:,k)>1);
    dofThdg = reshape(dghidx(tTh,:).',[],1);
    cg2dghT = cg2dgh(dofThdg,dofph);
    rhsp = (cg2dghT.'*Shdg(dofThdg,dofThdg))*P1dg(dofThdg,dgHidx(k,:));
    IHp = IH(dofpH,dofph);

    X = Sph\[IHp.' rhsp];
    mu = (IHp*X(:,1:length(dofpH)))\(IHp*X(:,length(dofpH)+(1:d+1)));
    CT{k} = sparse(Nh,d+1);
    CT{k}(dofph,:) = X(:,length(dofpH)+(1:d+1)) - X(:,1:length(dofpH))*mu;
end
```

其中 m 为补片内约束条件数量（粗网格内部节点数），远小于细网格自由度数，使 Schur 补矩阵规模极小。

6.4 粗网格刚度矩阵的组装与求解

将所有单元校正子叠加得到全局校正矩阵 `C_ell = cell2mat(CT) * cg2dgH`，多尺度基函数矩阵为 `G = P1 - C_ell`。粗网格 LOD 刚度矩阵通过三重矩阵乘积组装：

```
C_ell = cell2mat(CT)*cg2dgH;
G = P1 - C_ell;
dofH = true(NH,1);
dofH(TH.Dnodes) = 0;
G0 = G(:,dofH);
SHLOD0 = ((G0' * (cg2dgh' * Shdg)) * cg2dgh) * G0;
rhs = G0'*(cg2dgh'*(Mhdg*(cg2dgh*(P1*f(TH.p)))));
uH = zeros(NH,1);
uH(dofH) = SHLOD0\rhs;
uHms = G*uH;
```

粗网格系统求解后，通过 `uHms = G * uH` 重构细尺度信息。

为量化方法精度，程序进一步输出能量模与 L^2 模误差：

```
fprintf('LOD-Error (in energy norm): %7.5e\n',sqrt((uh-uHms).'*(Sh*(uh-uHms))));
fprintf('LOD-Error (in L2 norm): %7.5e\n',sqrt((uh-uHms).'*(Mh*(uh-uHms))));
fprintf('Error of FE-part of LOD (in L2 norm): %7.5e\n', ...
    sqrt((uh-P1*uH).'*(Mh*(uh-P1*uH))));
```

---

7 数值算例与应用分析

7.1 非周期高反差系数问题算例

验证算例采用具有高对比度随机棋盘格结构的扩散系数：在 $2^6 \times 2^6$ 的 Cartesian 网格上，每个格子独立以等概率取 $\alpha = 0.01$（白色，低传导）或 $\beta = 1$（黑色，高传导）。该系数既无周期性也无尺度分离，对数值方法构成严格测试。源项 $f \equiv 1$，边界条件为齐次 Dirichlet。

粗网格尺寸 H 取 $2^{-0.5}$ 至 $2^{-5.5}$，细参考网格固定为 $h = 2^{-8.5}$。局部化参数 $\ell = 0, 1, 2, 3, 4$。

7.2 收敛性验证

图 5.1 展示了能量模和 L^2 模误差随 H 的衰减曲线。标准有限元法（黑色虚线）在 H 较大时完全失效，误差始终保持在 $\mathcal{O}(1)$ 水平。LOD 方法当 ℓ 足够大时（如 $\ell \geq 3$）呈现清晰的线性（H^1 模）和二次（L^2 模）收敛，且误差常数与理论预测一致。当 ℓ 较小时，收敛曲线在某一 H 阈值后趋于水平，反映了局部化误差开始主导。

值得注意的是，即使取 $\ell = 0$（仅在一个单元内校正），LOD 解的有限元部分 $u_{H,\ell,h}$ 在 L^2 模下仍能逼近 V_H 中的最佳逼近，表明方法对参数选择具有一定鲁棒性。

7.3 结果可视化与解释

图2、图3与图4分别展示了细网格参考解、LOD 全近似解和 LOD 近似解中的有限元部分。可以看到，LOD 全近似解在整体形貌和局部震荡特征上与参考解高度一致；而有限元部分主要反映宏观包络信息。这一直观对比说明，LOD 通过多尺度基函数把细尺度信息有效编码到粗尺度离散空间中。

```
knitr::include_graphics("figures/fig6-reference-solution.png")
```

```
knitr::include_graphics("figures/fig6-lod-full-approximation.png")
```

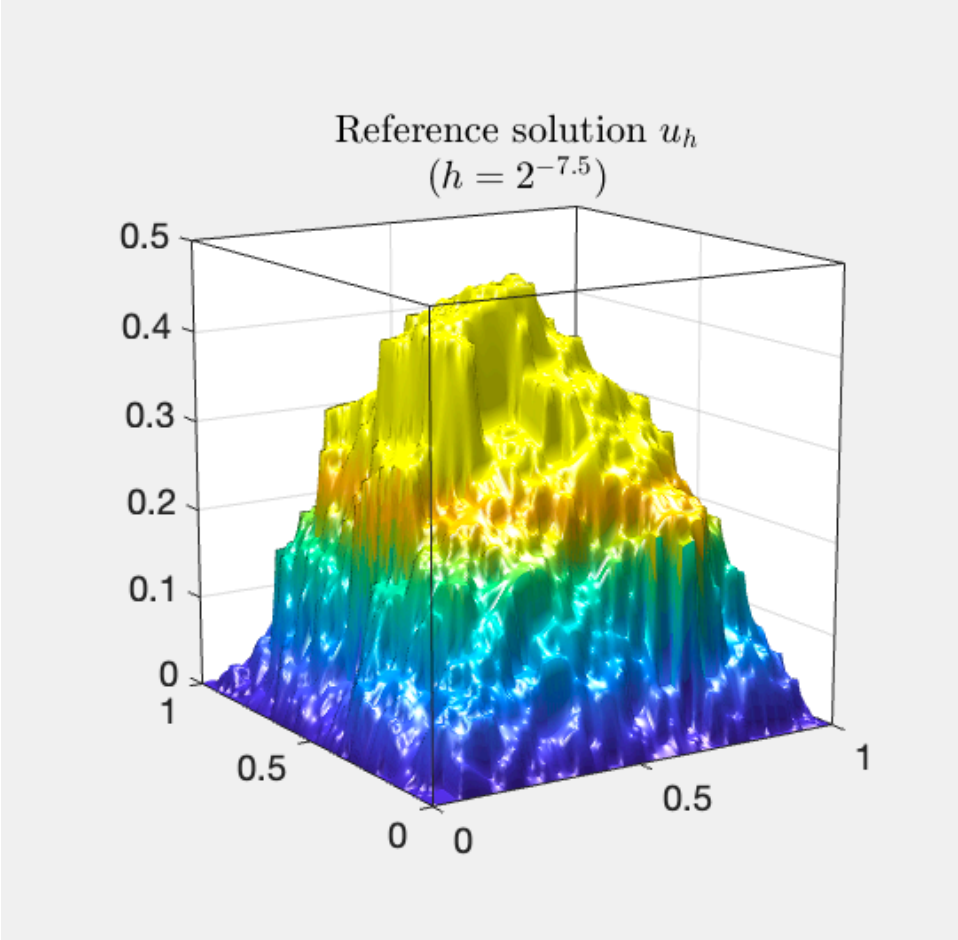

图 2: 细网格参考解 u_h（$h = 2^{-7.5}$）

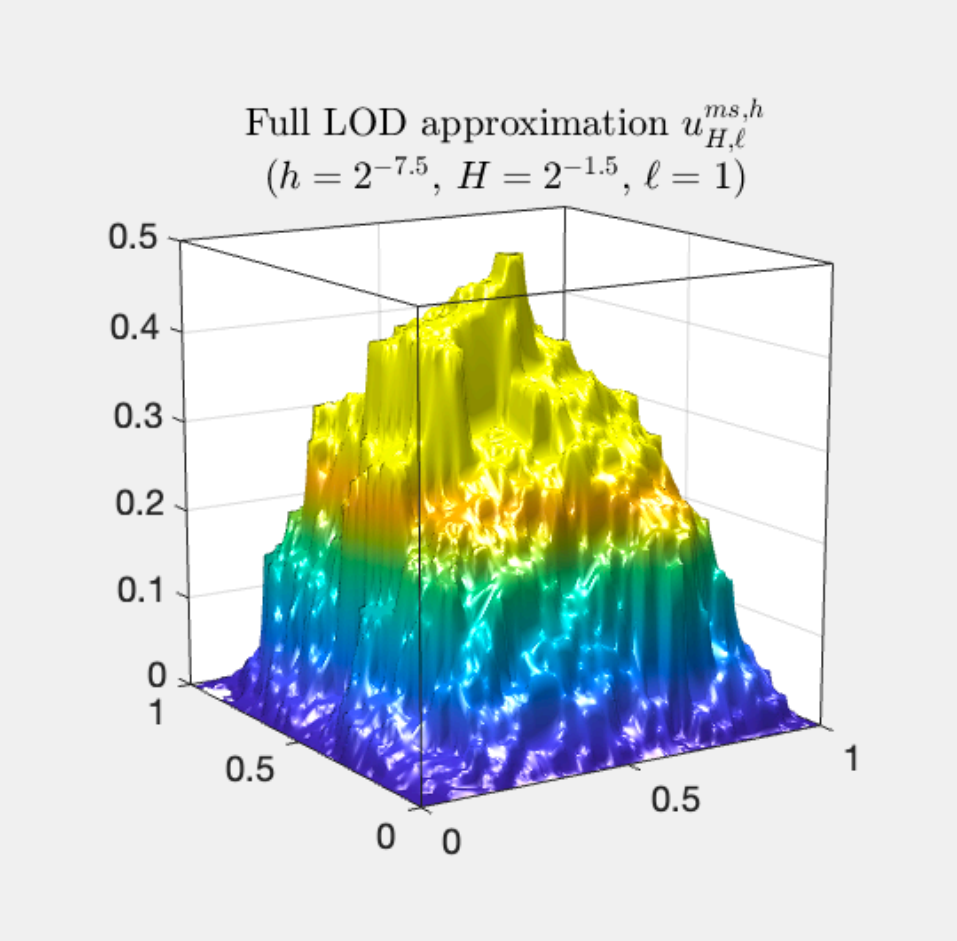

图 3: LOD 全近似解 $u_{H,\ell}^{ms,h}$（$h = 2^{-7.5}, H = 2^{-1.5}, \ell = 1$）

```r
knitr::include_graphics("figures/fig6-lod-fe-part.png")
```

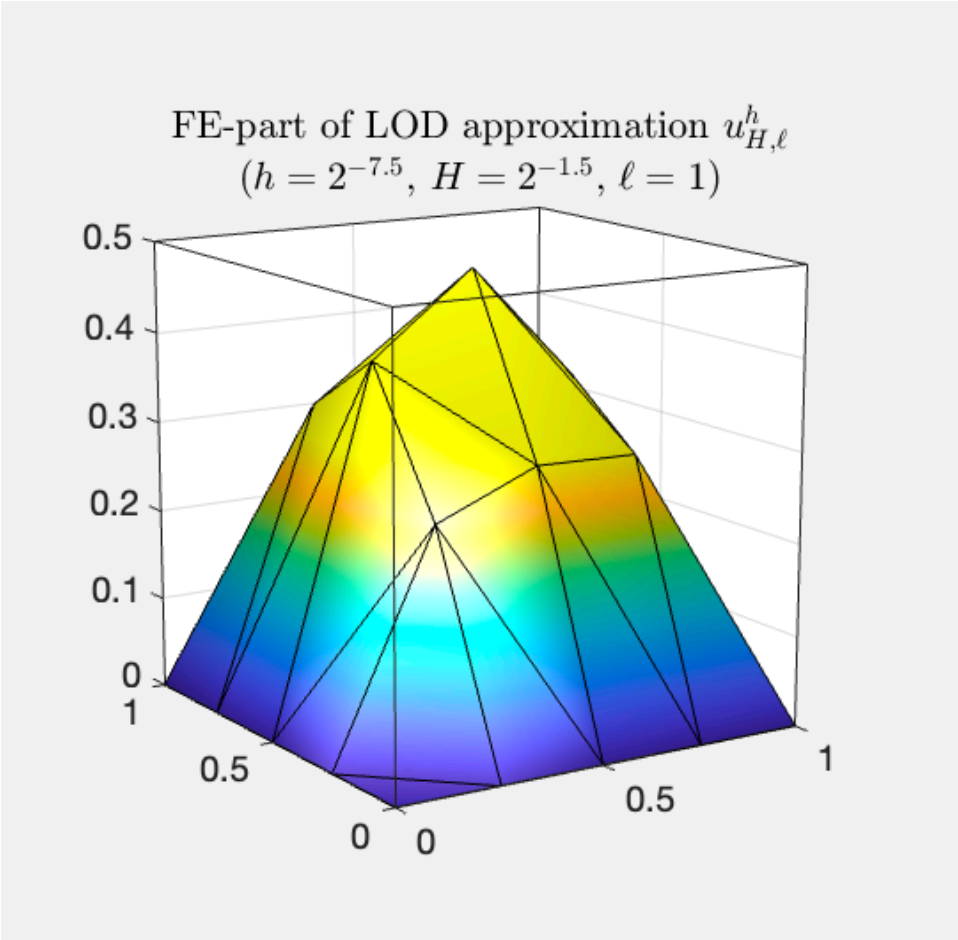

图 4: LOD 近似解中的有限元部分 $u^h_{H,\ell}$（$h = 2^{-7.5}, H = 2^{-1.5}, \ell = 1$）

---

8 结论与展望

8.1 本文总结

本文系统研究了求解二维椭圆型多尺度扩散问题的局部正交分解方法及其 MATLAB 程序实现。理论分析表明，LOD 方法通过拟插值算子实现尺度的稳定分解，进而通过 a-正交化构造具有微观信息编码能力的多尺度基函数。校正子的指数衰减性质为局部化近似提供了严格的数学基础，使得方法在保持最优收敛阶的前提下将计算复杂度降至与粗网格自由度成线性关系。

基于红化加密网格和鞍点格式校正子求解的 MATLAB 程序完整实现了 LOD 方法的全流程。数值实验验证了方法对非周期高反差系数问题的良好适应性，其收敛行为与理论预测一致，且精度显著优于相同计算代价下的标准有限元法。

8.2 未来研究方向

LOD 方法的理论框架具有良好的可扩展性。在应用层面，方法已被成功推广至线性与非线性特征值问题，在粗网格上实现了特征对的高精度逼近（特征值 H^4 收敛），进一步结合后处理技术可将收敛阶提升至 H^6。在抛物型问题中，离线构造的多尺度空间可在整个时间推进过程中复用，带来显著的计算效率增益。

在方法论层面，Petrov-Galerkin 版本的 LOD 方法通过使用标准有限元检验函数避免了校正子间内积的计算，进一步简化了程序实现并减少了通信开销。针对高反差系数问题的鲁棒插值算子设计也是活跃的研究方向。此外，LOD 方法在弹性力学、电磁波传播、随机同质化等领域的扩展应用正不断涌现，展示了其作为通用多尺度数值计算框架的巨大潜力。

---

9 附录：主要源程序清单

下列代码清单与主程序 `basicLOD2d.m` 保持一致，并通过 `code.md` 在论文中完整给出。

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%% Example code for Localized Orthogonal Decomposition method in 2d
%  Supplementry material of the book
%% An Introduction to the Localized Orthogonal Decomposition method
%  by
%% A. Malqvist (Chalmers University of Technology and University of Gothenburg)
%  and
%% D. Peterseim (University of Augsburg)
%
% The code is explained in detail in Chapter 7 of the book. It produces an
% LOD approximation of a prototypical linear elliptic diffusion problem as
% used throughout the book. The script can be run in Matlab or Octave.
% All required subroutines are contained in this file. The user may adapt
% the parameters.
%
% Copyright (C) 2020 Daniel Peterseim - All Rights Reserved
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%% Main script
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%% Clear work space
clear all;

%% Model problem
% physical dimension
d = 2;
% seed the random number generator (if possible)
if exist('rng') rng(27); end
% bounds of diffusion coefficient A
alpha = 0.01;
beta = 1;
% number of cells per coordinate direction
Neps = 2^6;
% diffusion coefficient on Cartesian mesh
Aeps = alpha+(beta-alpha).*(randi(2,Neps,Neps)-1);
% turn Aeps into function depending on (a list of) points in Omega
A = @(x) Aeps((ceil(x(:,1)*Neps)-1)*Neps+(ceil(x(:,2)*Neps)));
%A = @(x) 1;
% right-hand side
f = @(x) ones(size(x,1),1);

%% Initial mesh
% list of coordinates of vertices
p = [0 0; 1 0; 1 1; 0 1];
% triangles by references to vertex numbers
t = [1 2 4; 2 3 4];
% initial triangulation of Omega as structure array
T0 = struct('t',t,'p',p,'Dnodes',[1;2;3;4]);

%% LOD parameters
Hlevel =2;
hlevel = 8;
ell = 1;
```

```
%% coarse mesh
% refine initial mesh Hlevel times
[TH] = refineMesh(T0,Hlevel);
% define numbers of vertices NH and elements NTH
NH = size(TH.p,1);
NTH = size(TH.t,1);
% define number of local vertices
NHdg = numel(TH.t);
% enumerate local vertices acoording to their appearence in TH.t
dgHidx = reshape(1:NHdg,d+1,[]).';
% compute area of elements
areaTH = computeArea(TH);
% nngH is the number of neighbors for interior and 0 for boundary vertices
nngH = accumarray(TH.t(:),1,[NH,1]);
nngH(TH.Dnodes) = 0;

%% fine mesh
% refine initial mesh hlevel-Hlevel times
[Th,P1,R1,P0,P1dg] = refineMesh(TH,hlevel-Hlevel);
% define numbers of vertices Nh and elements NTh
Nh = size(Th.p,1);
NTh = size(Th.t,1);
% define number of local vertices
Nhdg = numel(Th.t);
% enumerate local vertices acoording to their appearence in Th.t
dghidx = reshape((1:Nhdg),d+1,[])';
% nngh is the number of neighbors for interior and 0 for boundary vertices
nngh = accumarray(Th.t(:),1,[Nh,1]);
nngh(Th.Dnodes) = 0;

%% quasi-interpolation
% embedding of continuous into discontinuous finite element functions
cg2dgh = sparse(1:Nhdg,reshape(Th.t.',1,[]),1,Nhdg,Nh);
% mass matrix for finite discontinuous finite elements
Mhdg = kron(spdiags(computeArea(Th),0,NTh,NTh),[2 1 1;1 2 1;1 1 2]./12);
% inverse of mass matrix for coarse discontinuous finite elements
B = kron(spdiags(1./computeArea(TH),0,NTH,NTH),[9 -3 -3;-3 9 -3;-3 -3 9]);
```

```
% compute L2 projection onto coarse discontinuous finite elements
PiHdg = B*((P1dg.'*Mhdg)*cg2dgh);
% embedding of continuous into discontinuous finite element functions
cg2dgH = sparse(1:NHdg,reshape(TH.t.',1,[]),1,NHdg,NH);
EH = spdiags(1./sum(cg2dgH,1).',0,NH,NH)*cg2dgH.';
% IH is the product of EH and PiHdg.
IH = EH*PiHdg;
%% for test
MHdg = kron(spdiags(computeArea(TH),0,NTH,NTH),[2 1 1;1 2 1;1 1 2]./12);

%% element patches
% incidence matrix of coarse elements and vertices
Ivt = sparse(TH.t,repmat((1:NTH).',1,d+1),1,NH,NTH);
% incidence matrix of coarse elements
Itt = spones(Ivt.'*Ivt);
% initialize patch matrix as identity
patch = speye(NTH);
% succesively apply Itt ell times
for k = 1:ell
    patch = Itt*patch;
end
% set non-zero entries to 1 and diagonal entries to 2
patch = spones(patch)+speye(NTH);

%% global stiffness and mass matrix
% evaluate diffusion coefficient at midpoints of elements
Ah = A((Th.p(Th.t(:,1),:)+Th.p(Th.t(:,2),:)+Th.p(Th.t(:,3),:))./3);
% assemble stiffness matrix for discontinuous functions
Shdg = assembleStiffnessMatrixDG(Th,Ah);

%% element correctors
% initialize cell array for element correctors
CT = cell(1,NTH);
% loop over coarse elements
for k=1:NTH
    % tpH(j) = true iff coarse element j is in ell-patch of element k
    tpH = logical(patch(:,k));
```

```
    % indices of corase patch vertices
    dofpH = find(accumarray(reshape(TH.t(tpH,:),[],1),1,[NH 1]));
    % remove (global) boundary vertices
    dofpH = dofpH(nngH(dofpH)>0);
    % tph(j) = true iff fine element j is in ell-patch of element k
    tph = logical(P0*patch(:,k));
    % nngph is the number of neighbors for fine patch vertices and 0 else
    nngph = accumarray(reshape(Th.t(tph,:),[],1),1,[Nh 1]);
    % non-zero entries of nngph identify fine patch vertices
    dofph = find(nngph);
    % remove boundary vertices by comparing number of neighbors
    % in the patch with number of global neighbors
    dofph = dofph(nngph(dofph)==nngh(dofph));
    % restrict fine stiffness matrix to dof
    dofphdg = reshape(dghidx(tph,:).',[],1);
    cg2dghk = cg2dgh(dofphdg,dofph);
    Sph = (cg2dghk.'*Shdg(dofphdg,dofphdg))*cg2dghk;
    % tTh(j) = true iff fine element j is in coarse element k
    tTh = logical(P0*patch(:,k)>1);
    % assemble right-hand side
    dofThdg = reshape(dghidx(tTh,:).',[],1);
    cg2dghT = cg2dgh(dofThdg,dofph);
    rhsp = (cg2dghT.'*Shdg(dofThdg,dofThdg))*P1dg(dofThdg,dgHidx(k,:));
    % restrict quasi interpolation to patch
    IHp = IH(dofpH,dofph);
    % solve corrector problem
    X = Sph\[IHp.' rhsp];
    mu = (IHp*X(:,1:length(dofpH)))\(IHp*X(:,length(dofpH)+(1:d+1)));
     CT{k} = sparse(Nh,d+1);
     CT{k}(dofph,:) = X(:,length(dofpH)+(1:d+1))-X(:,1:length(dofpH))*mu;
     %% for test
%       mH=length(dofpH);
%       mh=length(dofph);
%       MHdgT=MHdg(dofpH,dofpH);
%       X=[Sph (MHdgT*IHp).';
%          MHdgT*IHp  zeros(mH,mH)];
%        u=X\[rhsp;zeros(mH,3)];
```

```
%
%
%       X=[Sph -(MHdgT*IHp).';
%         MHdgT*IHp  zeros(mH,mH)];
%       Tran=cg2dgH(:,dofpH);
%    Tran=Tran';
%    dof4Ek=3*k-2:3*k;
%    rhs2=Tran*MHdg(:,dof4Ek);
%       rhs=[zeros(mh,3);rhs2];
%       u=X\rhs;
%       CT{k}(dofph,:)=u(1:mh,:);
%      CTT{k} = sparse(Nh,d+1);
%      CTT{k}(dofph,:) = u(1:mh,:);

end
% global correction operator
C_ell = cell2mat(CT)*cg2dgH;
% LOD basis
G = P1-C_ell;
%G=cell2mat(CT)*cg2dgH;

%% Build and solve coarse LOD system
% coarse degrees of freedom (interior vertices)
dofH = true(NH,1);
dofH(TH.Dnodes) = 0;
% LOD stiffness matrix
% assemble stiffness and mass matrix for continuous finite elements
G0 = G(:,dofH);
SHLOD0 = ((G0.'*(cg2dgh.'*Shdg))*cg2dgh)*G0;
% right-hand side
rhs = G0.'*(cg2dgh.'*(Mhdg*(cg2dgh*(P1*f(TH.p)))));
% solve coarse linear system
uH = zeros(NH,1);
uH(dofH) = SHLOD0\rhs;
% reconstruct fine-scale information to compute LOD approximation
uHms = G*uH;
```

```
%% Reference solution
dofh = true(Nh,1);
dofh(Th.Dnodes) = 0;
cg2dgh0 = cg2dgh(:,dofh);
Sh = (cg2dgh.'*Shdg)*cg2dgh;
Mh = (cg2dgh.'*Mhdg)*cg2dgh;
uh = zeros(Nh,1);
uh(dofh) = Sh(dofh,dofh)\(Mh(dofh,:)*f(Th.p));

%% Vizualization
% We shall finally plot the reference solution along with the LOD
% approximation.

%% test
% figure(1)
% huHh = trisurf(Th.t,Th.p(:,1),Th.p(:,2),uHms,'EdgeColor','none','FaceColor','interp');
% camproj perspective;campos([-3,-5,1]);daspect([1,1,0.5]);box on;set(gca,'BoxStyle','full');
% light;lighting phong;camlight('left');
% axis([0 1 0 1 0 0.5]);caxis([0 .3]);

figure
set(gcf, 'Position',  [10, 10, 1200, 300])
% plot coefficient
subplot(1,4,1)
[X,Y] = meshgrid(linspace(0,1,Neps),linspace(0,1,Neps));
hA = surf(X,Y,Aeps,'EdgeColor','none');
hold on; axis equal; view(0,90);colormap(gca,gray);caxis([alpha,beta]);
h = title({'Diffusion coefficient $A$',['($\alpha = ',num2str(alpha),'$ (black) , $\beta = ',num2str(
set(h,'Interpreter','latex')
% plot reference solution
subplot(1,4,2)
huh = trisurf(Th.t,Th.p(:,1),Th.p(:,2),uh,'EdgeColor','None');
camproj perspective;campos([-3,-5,1]);daspect([1,1,0.5]);box on;set(gca,'BoxStyle','full');
light;lighting phong;camlight('left');
axis([0 1 0 1 0 0.5]);caxis([0 .3]);
h = title({['Reference solution $u_h$'],['($h=2^{-',num2str(hlevel-0.5),'}$)']});
set(h,'Interpreter','latex')
```

```
% plot LOD approximation
subplot(1,4,3)
huHh = trisurf(Th.t,Th.p(:,1),Th.p(:,2),uHms,'EdgeColor','none','FaceColor','interp');
camproj perspective;campos([-3,-5,1]);daspect([1,1,0.5]);box on;set(gca,'BoxStyle','full');
light;lighting phong;camlight('left');
axis([0 1 0 1 0 0.5]);caxis([0 .3]);
h = title({'Full LOD approximation $u^{ms,h}_{H,\ell}$',['($h=2^{-',num2str(hlevel-0.5),'}$, $H=2^{-'
set(h,'Interpreter','latex')
% plot FE part of LOD approximation
subplot(1,4,4)
 huHh = trisurf(TH.t,TH.p(:,1),TH.p(:,2),uH,'EdgeColor','k','LineWidth',0.5,'FaceColor','interp');
camproj perspective;campos([-3,-5,1]);daspect([1,1,0.5]);box on;set(gca,'BoxStyle','full');
light;lighting phong;camlight('left');
axis([0 1 0 1 0 0.5]);caxis([0 .3]);
h = title({['FE-part of LOD approximation $u^{h}_{H,\ell}$'],['($h=2^{-',num2str(hlevel-0.5),'}$, $H=
set(h,'Interpreter','latex')
% % plot difference of LOD approximation and reference solution
% subplot(1,4,4)
% huHh = trisurf(Th.t,Th.p(:,1),Th.p(:,2),uh-uHms,'EdgeColor','none','FaceColor','interp');
% camproj perspective;campos([-3,-5,1]);daspect([1,1,0.5]);box on;set(gca,'BoxStyle','full');
% light;lighting phong;camlight('left');


%% Error
% Error of the LOD approximation in the energy norm
fprintf('LOD-Error (in energy norm): %7.5e\n',sqrt((uh-uHms).'*(Sh*(uh-uHms))));
% Error of the LOD approximation in the L^2 norm
fprintf('LOD-Error (in L2 norm): %7.5e\n',sqrt((uh-uHms).'*(Mh*(uh-uHms))));
% Error of FE-part og the LOD approximation in the L^2 norm
fprintf('Error of FE-part of LOD (in L2 norm): %7.5e\n',sqrt((uh-P1*uH).'*(Mh*(uh-P1*uH))));


%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%% Subroutines
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%% Function refineMesh
```

```
% refines a coarse triangulation TH nref times by applying the
% function REFINE nref times. Suitable restriction and prolongation
% operator between corresponding mesh functions are computed
function [Th,P1,R1,P0,P1dg] = refineMesh(Th,nref)
    % number of vertices
    NH = size(Th.p,1);
    % compute fine reference mesh
    P1 = speye(NH);
    P0 = speye(size(Th.t,1));
    P1dg = speye(numel(Th.t));
    for k=1:nref
        [Th,p,p0,p1dg] = refine(Th);
        P1dg = p1dg*P1dg;
        P1 = p*P1;
        P0 = p0*P0;
    end
    % compute coarse mesh nodal interpolant
    Nh = size(Th.p,1);
    [i,j] = find(P1>1-2*eps);
    R1 = sparse(j,i,ones(length(i),1),NH,Nh);
end
% End of function refineMesh

%% Function refine
% refines the current triangulation by dividing
% each marked element into 2^dim congruent elements (red refinement)
function [T,P,P0,P1dg] = refine(T)
    % Construct data structure
    [np] = size(T.p,1);
    nt = size(T.t,1);
    [e,nmbe] = computeEdges(T);
    ne = size(e,1);
    d2p = sparse(e,e(:,[2,1]),(1:ne)'*[1 1],np,np);
    % New nodes from the mid points of each edge
    newnode = 0.5.*(T.p(e(:,1),:)+T.p(e(:,2),:));
    P = sparse((1:ne)'*[1 1],e,.5,ne,np);
    T.p = [T.p; newnode];
```

```
    P = [speye(np);P];
    emarker = (np+1:np+ne)';
    p = [T.t,emarker(d2p(T.t(:,1)+np*(T.t(:,2)-1))),...
             emarker(d2p(T.t(:,2)+np*(T.t(:,3)-1))),...
             emarker(d2p(T.t(:,3)+np*(T.t(:,1)-1)))];
    T.t = [p(:,[1,4,6]);p(:,[4,2,5]);p(:,[6,5,3]);p(:,[4,5,6])];
    tnew2t = repmat(1:nt,1,4).';
    E = speye(nt);
    P1dg = [kron(E,[1 0 0;.5 .5 0;.5 0 .5]);kron(E,[.5 .5 0;0 1 0;0 .5 .5]);
            kron(E,[.5 0 .5;0 .5 .5;0 0 1]);kron(E,[.5 .5 0;0 .5 .5;.5 0 .5])];
    Dnodes = false(np,1);
    Dnodes(T.Dnodes) = true;
    T.Dnodes = find([Dnodes;Dnodes(e(:,1)) & Dnodes(e(:,2)) & nmbe==1]);
    P0 = sparse((1:length(tnew2t))',tnew2t,1,length(tnew2t),nt);
end
% End of function REFINE

%% Function computeEdges
% computes the edges of the triangulation T
function [e,nmbe] = computeEdges(T)
    np = size(T.p,1);
    e = [T.t(:,[1,2]); T.t(:,[1,3]); T.t(:,[2,3])];
    e = sort(e,2);
    d2p = sparse(e(:,1),e(:,2),1,np,np);
    [e1,e2,nmbe] = find(d2p);
    e = [e1,e2];
end
% End of function computeEdges

%% Function assembleStiffnessMatrixDG calculates stiffness matrix with
% respect to simplicial mesh Th with optional Th-piecewise constant scalar
% diffusion coefficient coeff
% discontinuous Galerkin
function [S] = assembleStiffnessMatrixDG(Th,coeff)
    % compute gradients of FE functions
    ve1 = Th.p(Th.t(:,3),:)-Th.p(Th.t(:,2),:);
    ve2 = Th.p(Th.t(:,1),:)-Th.p(Th.t(:,3),:);
```

```
    ve3 = Th.p(Th.t(:,2),:)-Th.p(Th.t(:,1),:);
    area2 = abs(ve3(:,2).*ve2(:,1)-ve3(:,1).*ve2(:,2));
    G(:,:,1)=[-ve1(:,2) ve1(:,1)]./(area2*[1 1]);
    G(:,:,2)=[-ve2(:,2) ve2(:,1)]./(area2*[1 1]);
    G(:,:,3)=[-ve3(:,2) ve3(:,1)]./(area2*[1 1]);
    c = coeff.*computeArea(Th);
    % local stiffness matrices
    S = zeros(size(Th.t,1),9);
    for j = 1:3
        for k = 1:3
            S(:,k+3*(j-1)) = c.*dot(G(:,:,j),G(:,:,k),2);
        end
    end
    % create global matrix
    I = (1:3).'*ones(1,3);
    ndof = numel(Th.t);
    idx = reshape(1:ndof,3,[]).';
    S = sparse(idx(:,reshape(I.',1,9)),idx(:,reshape(I,1,9)),S,ndof,ndof);
end
% End of function assembleStiffnessMatrix

%% Function computeArea
% calculates the area of the elements of the triangulation T
function v = computeArea(T)
    d12 = T.p(T.t(:,2),:)-T.p(T.t(:,1),:);
    d13 = T.p(T.t(:,3),:)-T.p(T.t(:,1),:);
    v = (d12(:,1).*d13(:,2)-d12(:,2).*d13(:,1))./2;
end
% End of function computeArea
```